

fiuba

algo3

Polimorfismo Refactorización

Pablo Suárez

psuarez@fi.uba.ar

Contexto

Definimos

Encapsulamiento: cada objeto es responsable de responder a los mensajes que recibe, sin que quien le envía el mensaje tenga que saber cómo lo hace

Polimorfismo: capacidad de respuesta que tienen distintos objetos de responder de maneras diferentes a un mismo mensaje

Hay consecuencias...

Polimorfismo y refactorización



Polimorfismo

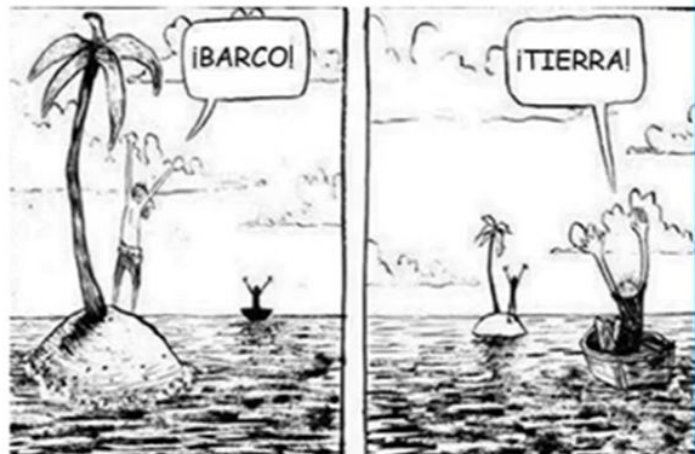
Capacidad que tienen distintos objetos de responder de maneras diferentes a un mismo mensaje

celda >> contiene(7)

fila >> contiene(7)

cajaAhorro >> extraer(100)

cuentaCorriente >> extraer(100)



Polimorfismo en lenguajes con clases

Distintos objetos responden de maneras diferentes a la llegada de un mensaje, basándose en la clase de la cual son instancias

cajaAhorro extraer: 100.

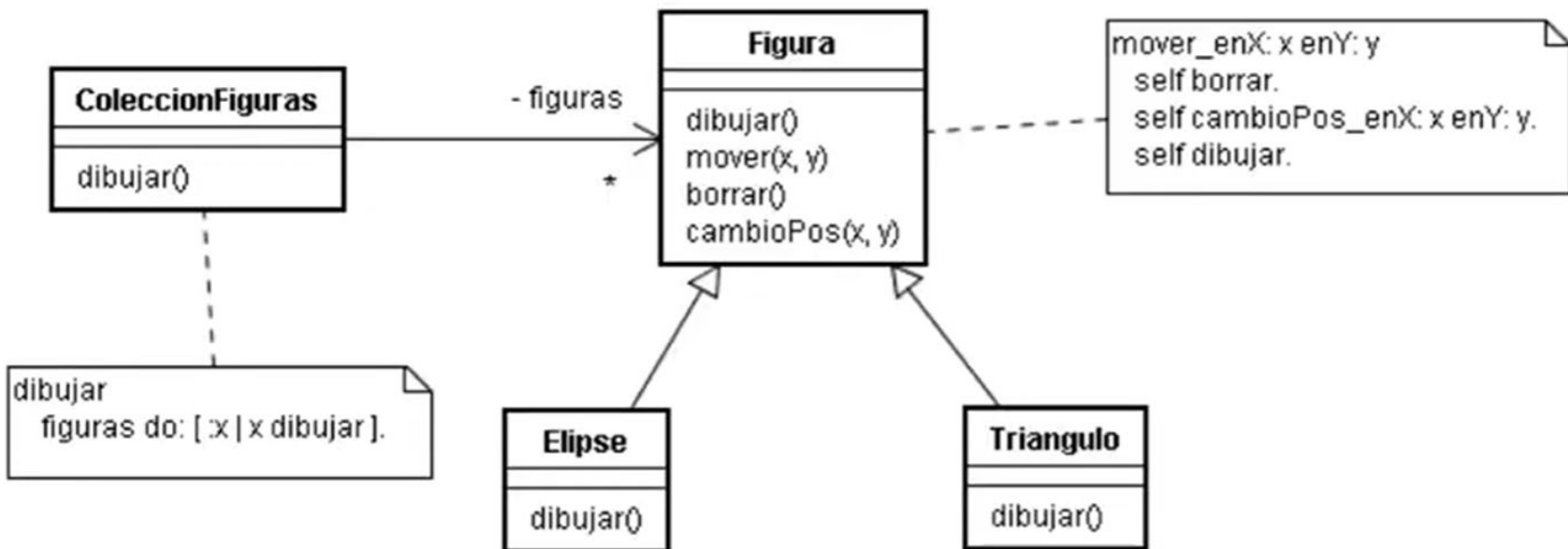
cuentaCorriente extraer: 100.



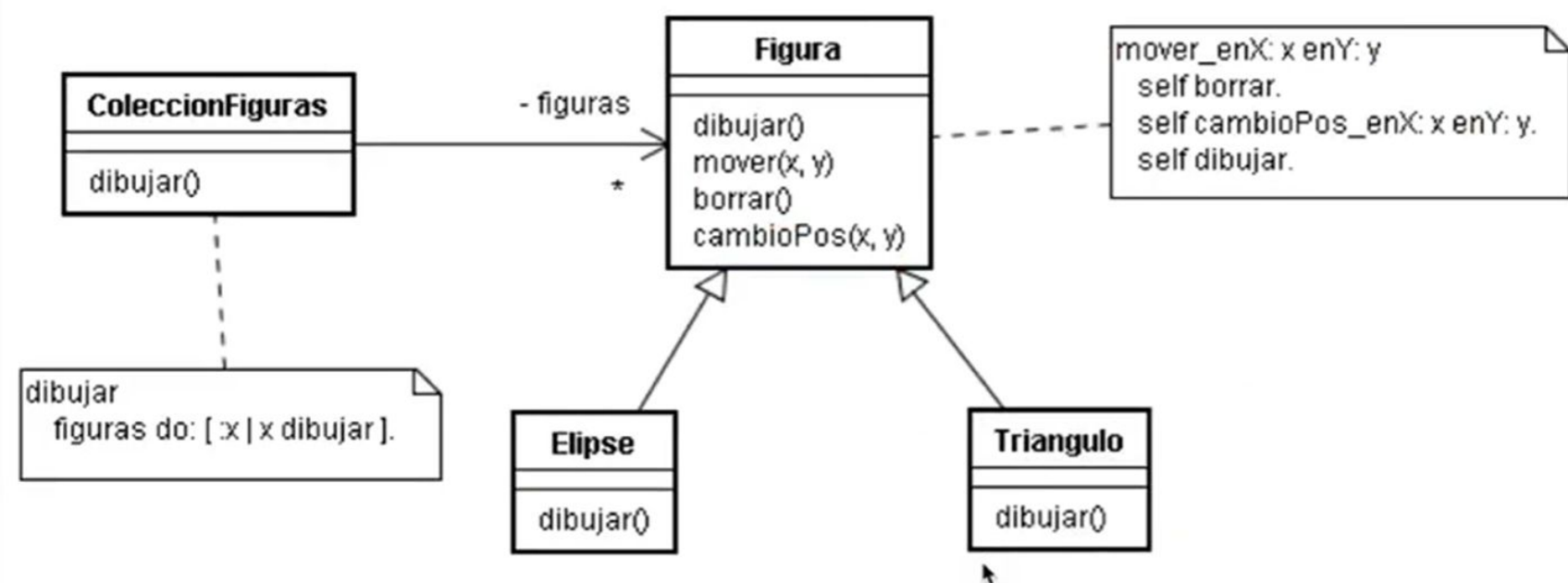
Polimorfismo: analizar en Smalltalk

fiuba

algo3

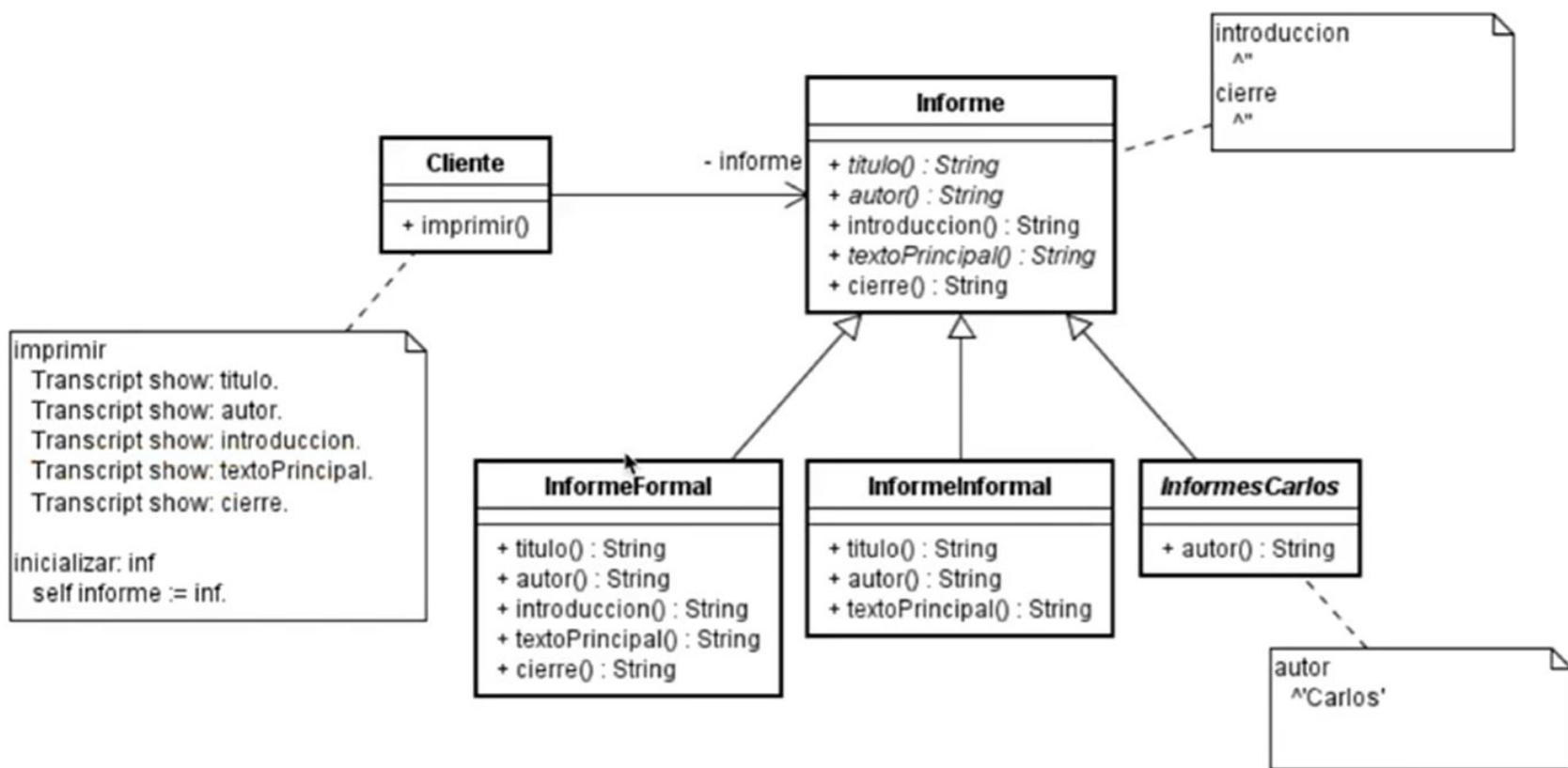


Polimorfismo: analizar en Java



¿Y si dibujar de Figura fuera abstracto? ¿Qué pasa con la verificación estática?

Polimorfismo: analizar



Algo un poco más complejo...

Ajedrez: cada tipo de pieza tiene un comportamiento distinto

=> Implementación obvia (?)

Coronación: ¿qué ocurre?

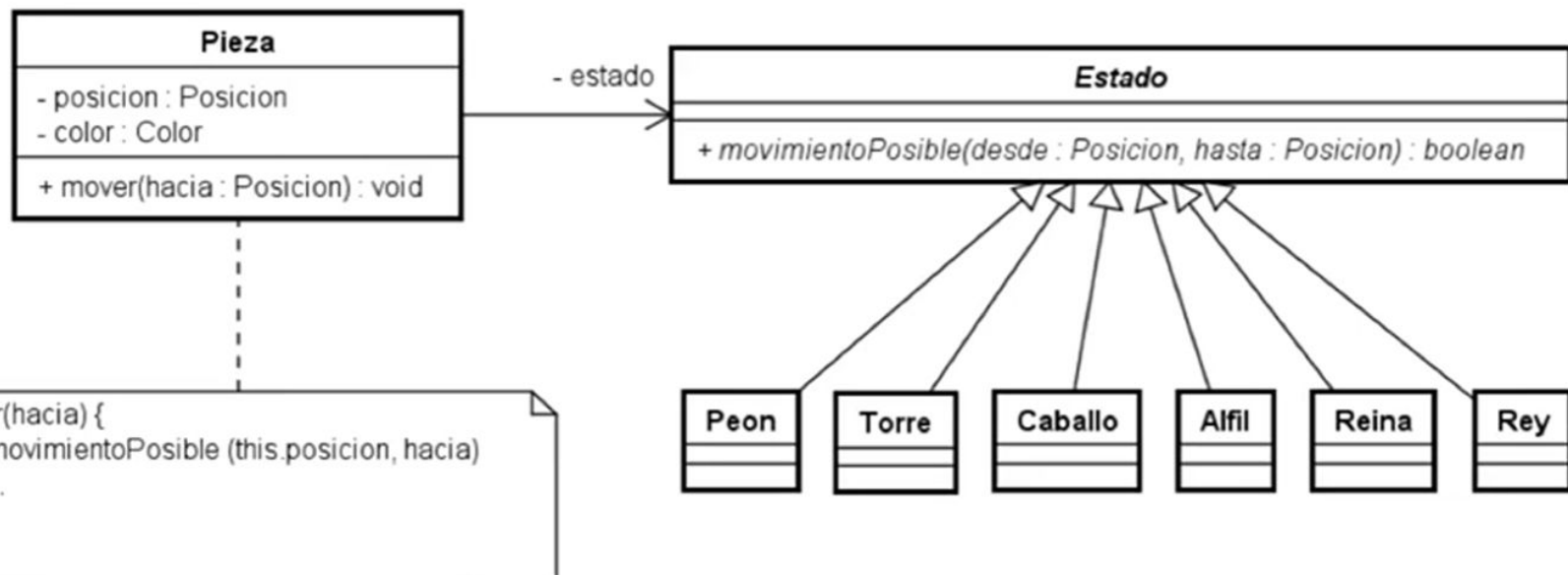
Posible interpretación: un objeto cambia de clase (su comportamiento varía luego de un cambio de estado)

... hay otros planteos: no es el mismo objeto

Quedémonos con el inicial: ¿puede un objeto cambiar su clase?



¿Y si vemos al tipo de pieza como un estado particular?



¿Qué hicimos?

Hemos encapsulado un método
(*movimientoPosible*) en un objeto

Podemos tratar a los métodos como objetos

En Smalltalk los bloques pueden ser objetos

En C# hay “delegados”

En Java 8, expresiones lambda



Soluciones típicas

Se las llama “patrones”

Usamos el patrón State en el ejercicio del ajedrez
Y el patrón Template Method en el ejemplo de los informes

Y algo parecido a Command o Strategy
en el ejemplo de la colección de figuras

Ya volveremos



Para pensar...



¿Vinculación tardía?

Se plantea que el polimorfismo funciona porque la decisión del método a invocar se toma en tiempo de ejecución y no antes
Tiene sentido en lenguajes de comprobación estática

```
cajaAhorro.extraer(100);  
cuentaCorriente.extraer(100);
```

En Java se da por defecto

Si no lo queremos, poner *final* al método

En C++ y C# se declaran métodos virtuales

Recapitulación



Recapitulación: preguntas

- ¿Por qué contraponemos el uso de “if” al polimorfismo?
- ¿Para qué querríamos polimorfismo sin herencia?
- ¿Polimorfismo es sinónimo de vinculación tardía?

